

Basket Catch - Signals, Groups, and Area2D

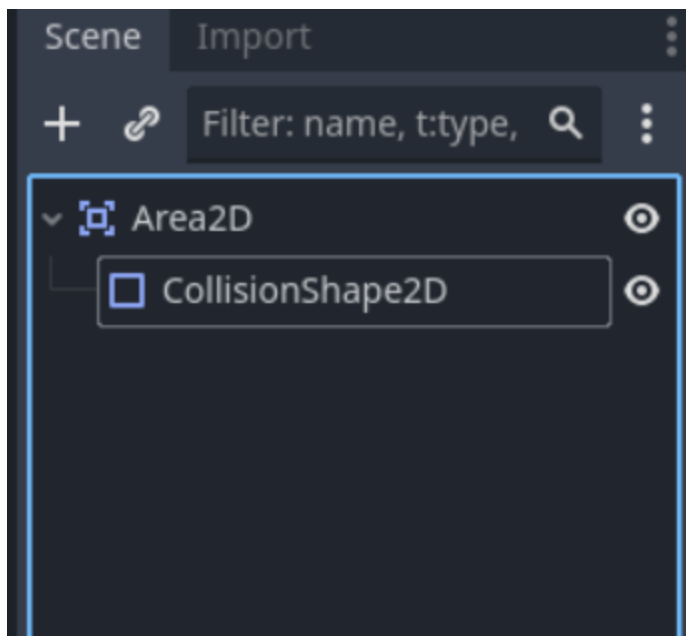
Some times we need to be able to tell if an object has entered a certain area or reached a certain place. To do this we use a Node called Area2D. This Node can communicate with other Nodes when an object enters or exits the Area2D. We may use Area2Ds to check if a player has:

- Touched a collectible
- Touched an obstacle
- Fallen off the stage
- Reached the stage goal
- Check for projectile collisions

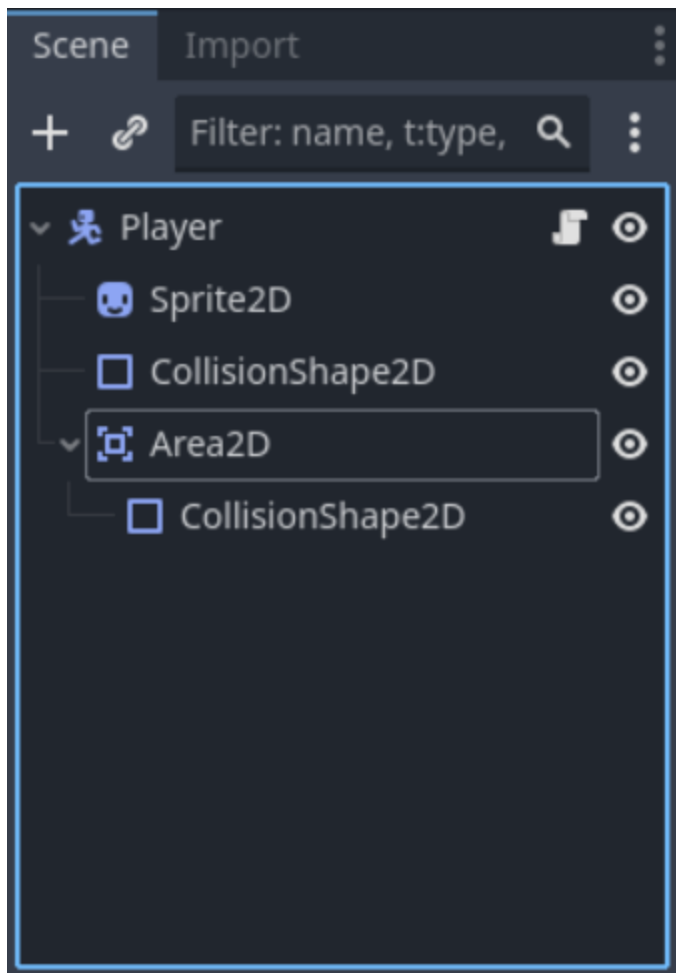
Area2Ds do this by sending out a *signal* when certain actions happen. Signals are Godot's way of allowing Nodes to communicate with each other *asynchronously* meaning we are not always receiving the signal. Think of a phone. We may receive a message from someone else when something important happens, but the person is most likely not messaging us with realtime updates on what they are doing. This is essentially how signals work. Let's dive in.

The Anatomy of an Area2D

Area2Ds are built similarly to our PhysicsBodies, needing a CollisionShape2D as a direct child to function. However, Area2D does not have any physics simulation applied to it, and will not stop PhysicsBodies from passing through it. In addition, Area2Ds are most commonly placed as a *child* of a root Node, not the root Node itself (though we can do this!). Below is what the scene tree of a properly constructed Area2D would look like (Note: This example uses the Area2D as the root but we will often see it as a child of the root Node):



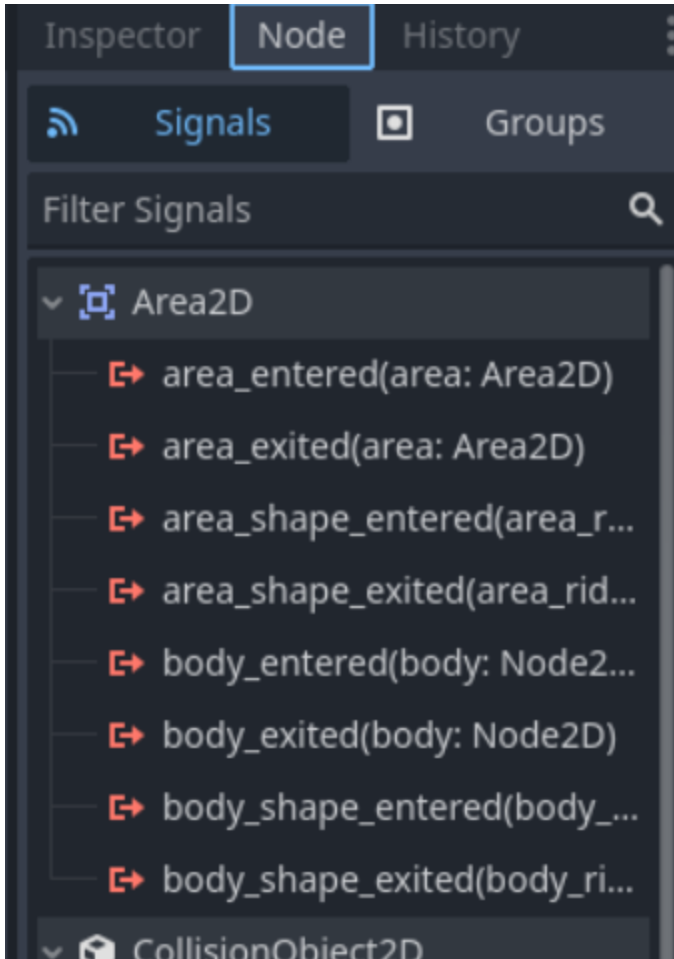
And here is an example of attaching an Area2D to a root node:



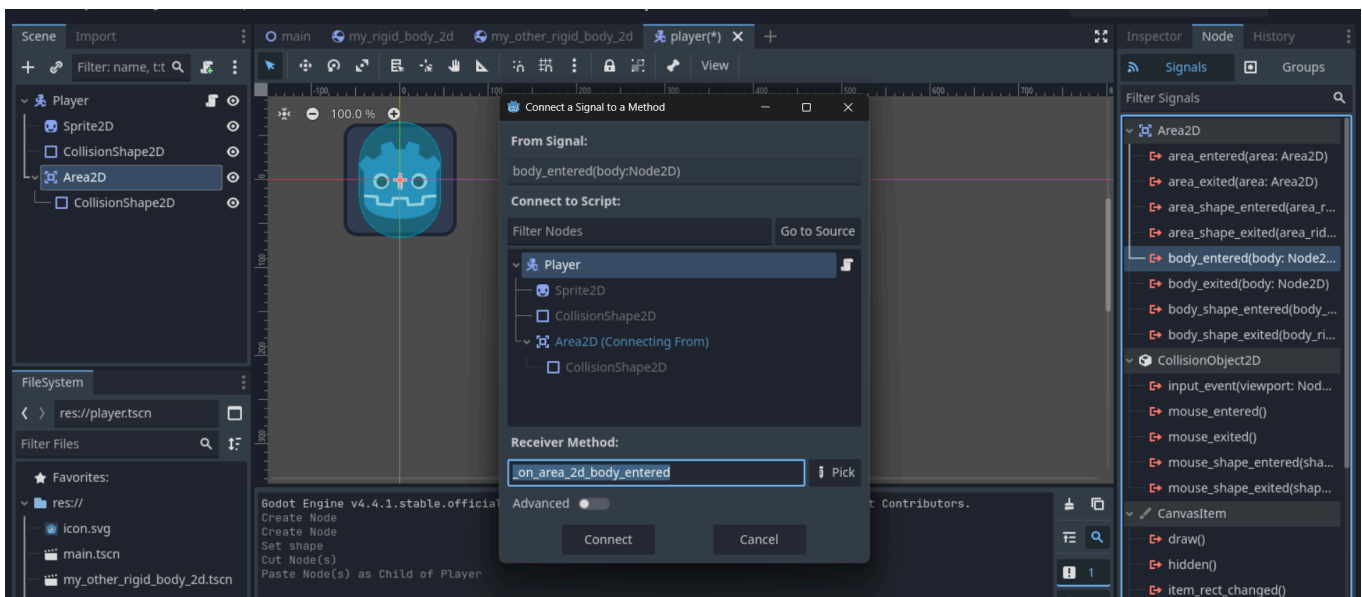
Connecting Signals

As stated above, signals are Godot's way of allowing Nodes to communicate with each other asynchronously. However, for a Node to be able to receive a signal it *must* have a script

attached to it. We can see the signals that a Node can *send* by looking at the "Node" tab in the Inspector Panel. Here are Area2D's signals:



We can connect a signal to another Node (remember this other Node must have a script attached to it) by double-clicking the signal we would like to send. This will open a dialogue box where we can choose the Node we want to send the signal to:



Once we select our target Node and click "Connect" a new *signal function* will be generated in the target Node's script:

```
extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = 800.0
var health = 100.0

func _physics_process(delta: float) -> void:

    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY * -1

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

# The function below was generated when we connected a signal
func _on_area_2d_body_entered(body: Node2D) -> void:
    pass # Replace with function body.
```

For this example we've selected the `body_entered` signal, which generated the `"on_area_2d_body_entered()"` function. This signal is the most common signal we will use with Area2Ds. We can see that the function declaration has an *argument* in its parentheses: "body". This argument is the PhysicsBody that is *entering* the Area2D's CollisionShape2D. We can manipulate the entering body by using dot notation with the "body" argument like below:

```
func _on_area_2d_body_entered(body: Node2D) -> void:

    # Moves the entering PhysicsBody 200 pixels right
    body.position.x += 200

    # Rotates the entering PhysicsBody 180 degrees (or 1PI radians)
    body.rotation = PI

    pass # Replace with function body.
```

We can also manipulate the object that the script is attached to (in this case the CharacterBody2D called "Player"):

```
func _on_area_2d_body_entered(body: Node2D) -> void:

    # Makes the Player "jump" up when they enter the area
    velocity.y -= 500

    # Reduces the health variable by 10 when the player enters the area
    health -= 10.0

    pass # Replace with function body.
```

Deleting Nodes

One of the most common uses of the Area2D's body_entered signal is to remove an object that enters the Area2D. We can easily do this by using the queue_free() method. Here's what that would look like in code:

```
func _on_area_2d_body_entered(body: Node2D) -> void:

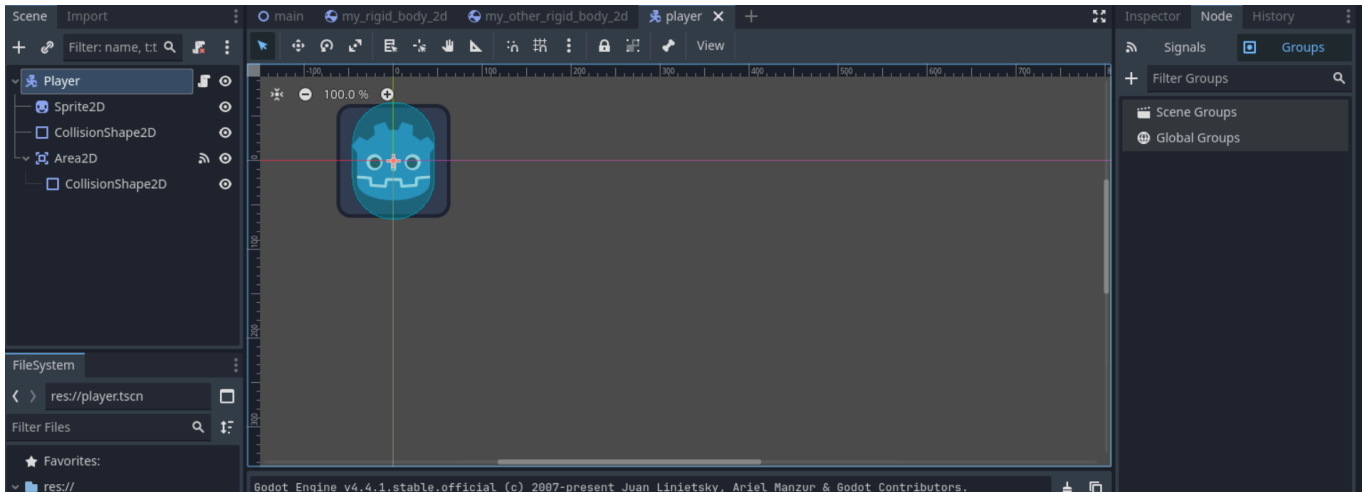
    # Deletes the entering body, removing it from the Scene
    body.queue_free()

    pass # Replace with function body.
```

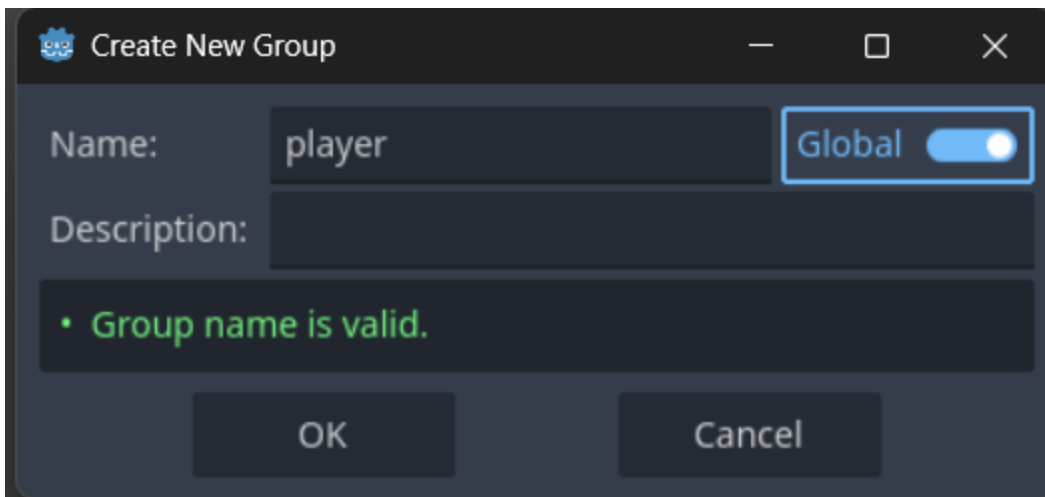
This code would wind up deleting *any* PhysicsBody that enters the Area2D, this would include the Player if their CollisionShape intersects the Area2D's CollisionShape. To avoid this, we can assign PhysicsBodies to a *group* and then check if the entering body is in the *group* we are looking for.

Groups

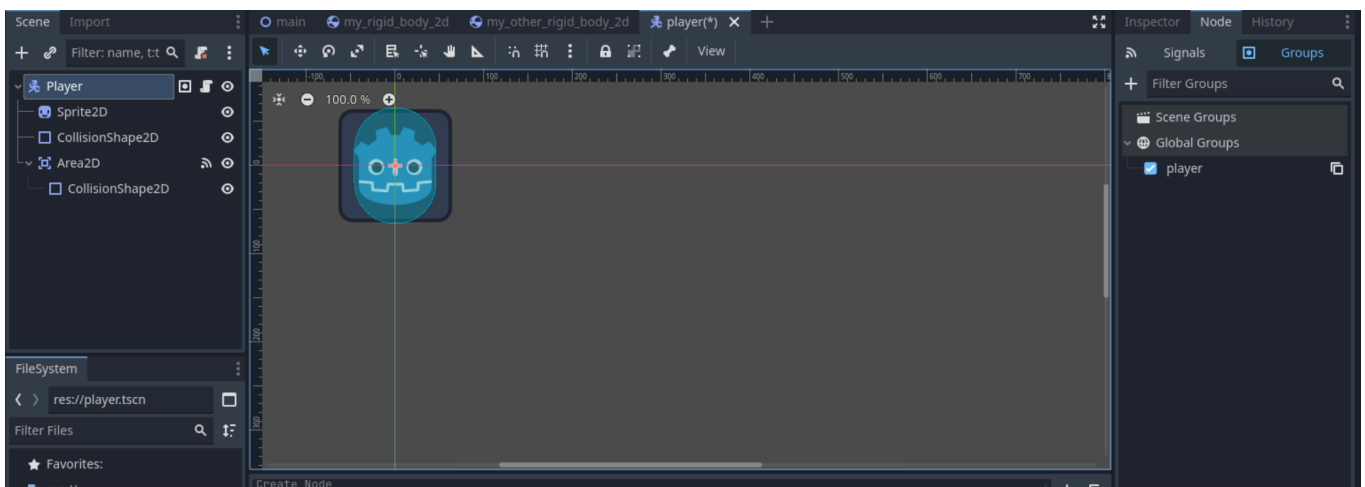
We add a Node to a group by selecting it in the scene tree and going to the "Groups" section of the "Node" tab:



We can create a new group by clicking the "+" button in the Groups section. This will open a dialogue box where we can name the group and select if it is a Global or Local group. (We will almost always select Global):



Once we hit "OK" the new group will be added to the "Groups" sections. We can then add a Node to the group by selecting the Node we want and checking the desired group's checkbox:



Checking if a Node is in a Group

In code, we can use a conditional to check if a Node is in a group. Below is the most common way we will do this for the `body_entered` signal:

```
func _on_area_2d_body_entered(body: Node2D) -> void:

    # Checks if the entering body is in the "player" group and deletes it if
    it is
    if body.is_in_group("player"):
        body.queue_free()

    pass # Replace with function body.
```

Now it's your turn!

In your Basket Catch project complete the following tasks:

1. Add your player to a new group called "player"
2. Add your falling objects to a new group called "collectibles"
3. Edit the CollisionShapes of your Player scene to allow your collectibles to fall through its center
4. Add an Area2D to the Player scene that is positioned at the *center* of your player.
5. Send the Area2D's "body_entered" signal to the Player scene's root node
6. Add code to the generated signal function that deletes collectibles when they touch the Area2D. Make sure this does not delete the player!